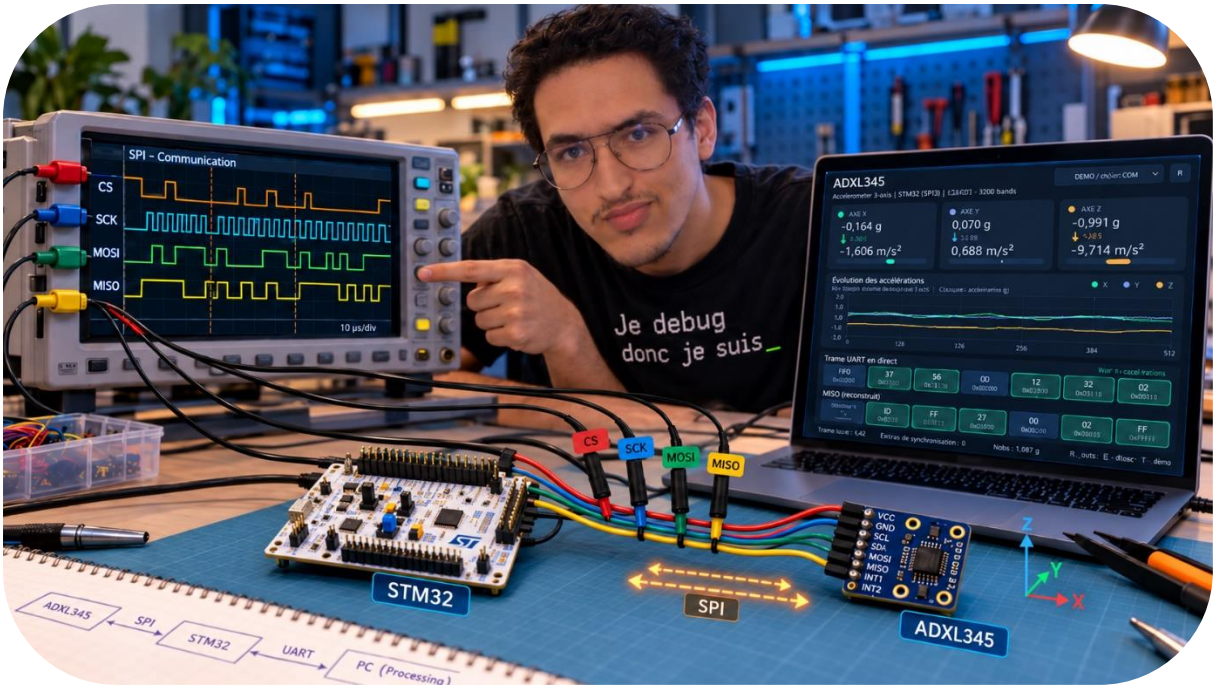
EL KATTOUFI Youssef

Année 2026 – 2027

TP 2 MCU



Etablissement scolaire : Polytech Sorbonne

Formation : Electronique et Informatique

Matières : Microcontrôleur 2

Enseignant :Francis BRAS

Table des matières

Remerciements 5

I. Introduction et objectif du TP 6

II. Arborescence du projet 7

III. Présentage du montage 8

IV. Bilan des entrées / sorties 9

A. Bilan E/S - Carte Nucleo STM32 F103RB 9

B. Raccordement de l'analyseur logique et parematrage de celui-ci 9

V. Explication du programme 10

A. Configuration des GPIO — GPIO_InitPin() 10

B. Configuration du SPI — config_SPI_1() et gere_cs() 10

C. Lecture et écriture des registres — config_regADXL(), lire_regADXL() et lire_multiple_regADXL() 11

D. Initialisation de l'accéléromètre — config_ADXL() 11

E. Récupération des accélérations — ADXL_ReadAxes() 12

F. Gestion des mesures et de la FIFO 12

G. Création et transmission des trames UART — FIFO_WriteADXLFrame() et FIFO_ServiceUSART2() 13

H. Fonctionnement général — main() 14

VI. Configuration des périphériques du STM32 15

A. Configuration GPIO 15

B. Configuration UART2 15

C. configuration SPI1 16

VII. Configuration des registres ADXL345 17

VIII. Processing avec l'ADXL345 17

IX. Chronogrammes de l'analyseur logique 18

A. SPI 18

B. USART 20

1. Exemple trame USART 20
2. Calcul de la durée de l'impulsion 20
X. Difficultés rencontrées et assistance IA 21
XI. Dessin demandé 21
XII. Table des figures 22

Remerciements

Je tiens tout d'abord à remercier Monsieur Francis BRAS pour avoir conçu et mis en place un TP aussi stimulant, qui nous a amenés à réfléchir, à nous questionner et à surmonter de nombreuses difficultés techniques.

Je souhaite également le remercier pour sa disponibilité et son accompagnement tout au long du projet, notamment à travers ses échanges sur Discord, où il a toujours su nous apporter son aide et ses conseils lors des phases de programmation.

Enfin, je le remercie pour la mise à disposition du matériel nécessaire à la réalisation de ce projet, ainsi que pour la transmission des connaissances en Microcontrôleur 2.

Son investissement et son suivi nous ont permis de mettre en pratique nos connaissances tout en développant notre autonomie et notre capacité à résoudre des problèmes techniques.

I. Introduction et objectif du TP

Dans ce TP, le but était de mettre en place une chaîne complète d'acquisition de mesures à partir d'un accéléromètre ADXL345, puis d'exploiter ces données avec un STM32F103RB et un affichage sur ordinateur.

L'idée était donc de récupérer les valeurs d'accélération sur les axes X, Y et Z, de comprendre les trames échangées avec le capteur, puis d'envoyer les données vers le PC pour pouvoir les visualiser clairement.

Pour la maquette, j'ai travaillé avec le STM32F103RB relié à l'ADXL345 en SPI, afin de lire les registres utiles du capteur et reconstruire les mesures. Ensuite, les données étaient transmises en UART vers l'ordinateur, où un programme Processing permettait d'afficher les valeurs mesurées, les courbes d'évolution et les octets de la trame reçue.

Le but n'était pas juste d'avoir une valeur affichée, mais aussi de bien voir comment les infos circulent entre le capteur, la carte STM32 et le PC.

J'ai réalisé les essais progressivement : d'abord la prise en main de la communication SPI avec le capteur, puis la lecture des registres de l'ADXL345, la récupération des données sur les trois axes, l'envoi des mesures en UART, et enfin la visualisation sur Processing.

J'ai aussi utilisé l'analyseur logique pour observer les signaux et vérifier que les trames échangées correspondaient bien à ce qui était attendu.

Pour le développement, j'ai utilisé Visual Studio Code pour écrire et modifier le code en C, puis Keil µVision pour le compiler, vérifier les éventuelles erreurs et enfin téléverser le programme sur la carte STM32F103RB.



Figure 1 : Visualisation Keil µVision + Visual Studio Code

Remarque :

Comme à mon habitude le projet se trouve sur mon compte github personnel à l'adresse suivante : https://github.com/Darkzveller/TP2_MCU

II. Arborescence du projet

Voici l'organisation du dossier TP_2 d'après mon inventaire du 18/10/2026. J'ai regroupé les sous-dossiers volumineux pour que l'ensemble reste lisible.

Texte brut

```
TP_2/
|
├── Projet_keil_initialiser_STM32_F103RB-main/    # Projet STM32 en C sous Keil
├── Visualisation_ADXL345/                        # Interface Processing et export Windows
├── Session_pulse_view/                           # Sessions et captures de l'analyseur logique
├── Piece_presentation_rapport/                   # Dossier des éléments de présentation du rapport
|
├── ADXL345.h                                      # Définitions des registres ADXL345
├── ADXL345.pdf                                    # Documentation de l'accéléromètre
├── TP2.pdf                                        # Sujet du TP au format PDF
├── TP2.docx                                       # Sujet du TP au format Word
├── Tutoriel_prise_en_main.pdf                     # Tutoriel de prise en main
├── Tutoriel_USART.pdf                             # Tutoriel sur la liaison série
├── utiliser lanalyseur logique 8ch 24MHz-20260925.zip
|                                                  # Documentation / ressources de l'analyseur
├── Branchement_analyzer_to_mpu.xlsx               # Correspondance des fils et voies analysées
├── preuve.pptx                                    # Présentation regroupant des éléments de preuve
|
├── Nucleo_Schematic.pdf                           # Schéma de la carte NUCLEO
├── User manuel UM1724 NUCLEO-F103RB.pdf           # Manuel de la carte NUCLEO
├── RESUME_STM32F103xx.pdf                         # Résumé de la documentation STM32F103
├── stm32f103xx_UM.pdf                             # Manuel de référence STM32
├── stm32f103errata.pdf                            # Errata du microcontrôleur
|
├── Visualisation_ADXL345.pdez                     # Archive du sketch Processing
├── TP2_MCU2.docx                                  # Compte rendu modifiable
├── TP2_MCU2.pdf                                   # Compte rendu exporté en PDF
├── TP_2_micro_controleur2.gsheet                  # Raccourci Google Sheets
├── git_push.bat                                   # Script local de commandes Git
├── .gitignore                                     # Exclusions Git
└── README.md                                      # Présentation et mode d'emploi du projet
```

III. Présentage du montage

Pour le montage, j'ai utilisé la carte **NUCLEO STM32-F103RB** avec l'accéléromètre **ADXL345**, installé sur une plaque Labdec. J'ai également branché les voies de l'analyseur logique sur les mêmes lignes que celles utilisées pour la communication SPI, ce qui permet d'observer les échanges sans modifier le fonctionnement du montage.

Pour gagner du temps, j'ai commencé à câbler le TP2 pendant que je travaillais encore sur le TP1. Je faisais donc les deux TP en parallèle, ce qui explique pourquoi certaines photos peuvent montrer des fils ou des composants qui ne sont pas directement liés à ce montage.

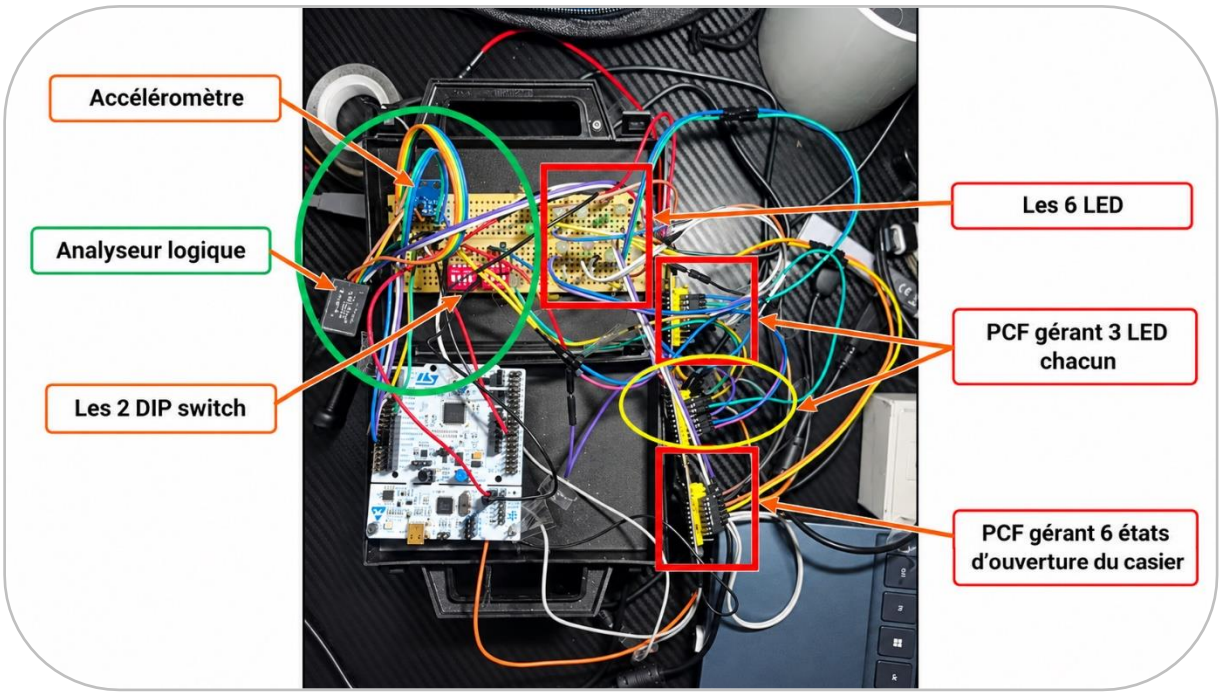


Figure 2 : Présentation du câblages

IV. Bilan des entrées / sorties

A. *Bilan E/S - Carte Nucleo STM32 F103RB*

Nom module	Nom signal	in/out/ ana/num	Pull up / Pull down	Périphérique	PX_n	Commentaires
COM_ORDI	GPIOA, 2 (TX)	OUT / Num	None	USART2	PA2	Envoi des données vers Processing
COM_ORDI	GPIOA, 3 (RX)	IN / Num	None	USART2	PA3	Réception UART configurée, mais non exploitée dans le programme actuel
ADXL345	/CS	OUT / Num	None	GPIOA	PA4	Sélection du capteur (actif à 0). Voie CH2 de l'analyseur logique.
ADXL345	SCK	OUT / Num	None	SPI1	PA5	Horloge du SPI en mode 3. Voie CH1 de l'analyseur logique.
ADXL345	MISO	IN / Num	None	SPI1	PA6	Données envoyées par le capteur vers le STM32. Voie CH4.
ADXL345	MOSI	OUT / Num	None	SPI1	PA7	Données envoyées par le STM32 vers le capteur. Voie CH3.
ADXL345	INT1	IN / Num	None	GPIOA	PA10	Entrée WATERMARK : FIFO du capteur à moitié remplie. Voie CH6.
ADXL345	INT2	IN / Num	None	GPIOA	PA11	Entrée OVERRUN : débordement FIFO. Voie CH5.

Figure 3 : Bilan E/S - Carte Nucleo STM32 F103RB

B. *Raccordement de l'analyseur logique et parematrage de celui-ci*

Les voies indiquées ci-dessous proviennent de l'expérience. La masse de l'analyseur doit être commune à celle du capteur et du STM32.

Canal PulseView	Signal observé	Interprétation
D0 / CH1	SCLK	Cadencement des bits SPI
D1 / CH2	/CS	Début et fin de transaction
D2 / CH3	MOSI	Commande et adresse envoyées au capteur
D3 / CH4	MISO	État des registres et octets de mesure
D4 / CH5	INT2	Alerte de débordement
D5 / CH6	INT1	Seuil WATERMARK atteint

Figure 4 : Tableau de raccordement analyseur logique

V. Explication du programme

J'ai volontairement séparé le programme en petites fonctions. Cela permet de tester chaque partie seule avant de tout rassembler, et cela évite de répéter les mêmes instructions dans le main().

A. Configuration des GPIO — GPIO_InitPin()

La fonction GPIO_InitPin() permet de configurer les broches du STM32 en fonction de leur utilisation. Elle active d'abord l'horloge du port concerné, puis modifie les registres CRL ou CRH pour définir le mode de fonctionnement de la broche.

Dans ce TP, elle est notamment utilisée pour configurer les broches nécessaires au SPI ainsi que les deux entrées permettant de surveiller les interruptions de l'ADXL345.

J'ai également utilisé GPIO_ReadPin() pour connaître l'état logique d'une broche. Cette fonction lit le registre IDR et renvoie un état haut ou bas. Elle sert principalement à surveiller les signaux INT1 et INT2 du capteur.

B. Configuration du SPI — config_SPI_1() et gere_cs()

La fonction config_SPI_1() initialise le périphérique SPI1 du STM32. J'ai configuré la communication en mode maître, avec des échanges sur 8 bits et une transmission du bit de poids fort en premier.

Le SPI fonctionne en mode 3, ce qui signifie que l'horloge est au niveau haut au repos (CPOL = 1) et que les données sont échantillonnées sur le deuxième front (CPHA = 1). Le diviseur d'horloge est réglé sur 64, ce qui donne une fréquence SPI de 1,125 MHz avec une horloge de 72 MHz.

Pour sélectionner l'accéléromètre, j'utilise la broche PA4 comme Chip Select (CS). La fonction gere_cs() permet de la faire passer à 0 pour commencer une communication, puis à 1 lorsque l'échange est terminé.

Les trois autres lignes SPI sont gérées directement par le périphérique SPI1 du STM32 : PA5 pour SCK, PA6 pour MISO et PA7 pour MOSI.

C. *Lecture et écriture des registres — config_regADXL(), lire_regADXL() et lire_multiple_regADXL()*

Ces trois fonctions permettent de communiquer directement avec les registres internes de l'ADXL345.

La fonction config_regADXL() sert à écrire une valeur dans un registre du capteur, par exemple pour modifier sa plage de mesure ou activer une fonctionnalité.

À l'inverse, lire_regADXL() permet de récupérer la valeur d'un seul registre, tandis que lire_multiple_regADXL() permet d'en lire plusieurs à la suite sans interrompre la communication SPI.

Pour chaque échange, le STM32 commence par envoyer un octet contenant l'adresse du registre ainsi que les bits indiquant s'il s'agit d'une lecture ou d'une écriture. Dans le cas d'une lecture multiple, le bit MB est également activé.

Par exemple, pour lire les accélérations à partir du registre DATAX0, situé à l'adresse 0x32, la commande envoyée est 0xF2. Elle permet ensuite de récupérer les six octets correspondant aux mesures X, Y et Z.

Cette fonction est particulièrement intéressante à observer avec l'analyseur logique, puisqu'on peut retrouver la commande envoyée sur MOSI et les octets renvoyés par le capteur sur MISO.

D. *Initialisation de l'accéléromètre — config_ADXL()*

La fonction config_ADXL() regroupe les différentes configurations nécessaires pour faire fonctionner l'accéléromètre.

J'ai notamment configuré le capteur en mode mesure, avec une plage de ±16 g et une résolution complète (*FULL_RES*). La fréquence d'acquisition est réglée à **100 Hz** grâce au registre BW_RATE.

Le programme configure également plusieurs paramètres liés à la détection d'activité, d'inactivité, de choc et de chute libre.

Pour gérer les mesures, j'utilise la FIFO interne de l'ADXL345 en mode *Stream*. Le seuil WATERMARK est fixé à 16 mesures. Cela permet au capteur de signaler qu'un certain nombre de mesures sont disponibles dans sa mémoire.

Enfin, les signaux WATERMARK et OVERRUN sont associés respectivement aux sorties INT1 et INT2 du capteur. Ces deux signaux sont ensuite surveillés par le STM32.

E. Récupération des accélérations — ADXL_ReadAxes()

Une fois le capteur configuré, la fonction ADXL_ReadAxes() permet de récupérer les mesures des trois axes.

Pour cela, elle appelle lire_multiple_regADXL() afin de lire six registres consécutifs, de DATAX0 à DATAZ1.

Chaque accélération est stockée sur deux octets : un octet de poids faible et un octet de poids fort. Le programme les rassemble pour obtenir une valeur signée de type int16_t.

Par exemple, pour l'axe X, l'octet DATAX1 est décalé de 8 bits vers la gauche, puis combiné avec DATAX0. Le même principe est ensuite appliqué aux axes Y et Z.

Le fait d'utiliser des valeurs signées est important, puisque l'accélération peut être positive ou négative suivant l'orientation du capteur.

F. Gestion des mesures et de la FIFO

Pour éviter de perdre des données pendant leur transmission, le programme utilise deux FIFO différentes : celle intégrée à l'ADXL345 et une seconde FIFO logicielle de 32 octets, appelée fifo_tx.

Lorsque le signal WATERMARK est détecté sur PA10, le STM32 lit le registre FIFO_STATUS afin de connaître le nombre de mesures disponibles dans le capteur.

Si au moins une mesure est présente et qu'il reste suffisamment de place dans la FIFO logicielle, le programme récupère les trois accélérations X, Y et Z.

Ces valeurs sont ensuite préparées pour leur transmission vers l'ordinateur.

La FIFO logicielle fonctionne de manière circulaire. Lorsqu'un indice atteint la fin du tableau, il revient au début. Cela permet de continuer à stocker et transmettre les octets sans avoir à déplacer toutes les données.

Le programme surveille également le signal OVERRUN sur PA11, même si aucun traitement particulier n'est encore réalisé lorsqu'il est détecté.

G. *Création et transmission des trames UART — FIFO_WriteADXLFrame() et FIFO_ServiceUSART2()*

Pour envoyer les mesures vers Processing, j'ai configuré l'USART2 à **9600 bauds au format 8N1**, grâce à la fonction config_usart2().

Avant l'envoi, la fonction ADXL_Split13Bits() conserve les 13 bits de chaque mesure, puis les sépare en deux octets : 6 bits de poids fort et 7 bits de poids faible.

La fonction FIFO_WriteADXLFrame() rassemble ensuite les données dans une trame de 7 octets, organisée de la manière suivante :

Octet	Contenu	
1	**0xF0 :** en-tête	En-tête de trame
2	XH : bits de poids fort de X	
3	XL : bits de poids faible de X	
4	YH : bits de poids fort de Y	
5	YL : bits de poids faible de Y	
6	ZH : bits de poids fort de Z	
7	ZL : bits de poids faible de Z	Fin de trame / dernier octet

La trame commence par l'octet d'en-tête **0xF0** et se termine par l'octet **ZL**.

Figure 5 ; Format de la trame UART

L'en-tête 0xF0 permet à Processing de reconnaître le début d'une nouvelle trame. Les six octets suivants contiennent les trois mesures d'accélération.

Les octets sont d'abord stockés dans fifo_tx, puis la fonction FIFO_ServiceUSART2() les transmet progressivement. Elle vérifie que le registre d'émission de l'USART est disponible avant d'envoyer l'octet suivant.

Cela évite de bloquer systématiquement le programme en attendant la fin de chaque transmission.

H. Fonctionnement général — main()

La fonction main() commence par initialiser la communication série avec l'ordinateur, puis appelle config_ADXL() pour préparer le SPI et configurer l'accéléromètre.

Une fois les initialisations terminées, le programme entre dans une boucle infinie.

À chaque passage, il essaie d'abord de transmettre un octet présent dans la FIFO logicielle. Il vérifie ensuite si le signal WATERMARK est actif.

Lorsque des mesures sont disponibles et qu'il reste de la place pour une trame complète, il lit l'état de la FIFO du capteur, récupère une mesure X/Y/Z et prépare les sept octets destinés à Processing.

Enfin, il vérifie également l'état du signal OVERRUN.

Le programme fonctionne donc en continu, sans avoir besoin de redémarrer une acquisition à chaque mesure. Les données sont récupérées au fur et à mesure, puis envoyées à l'ordinateur pour être reconstituées et affichées.

VI. Configuration des périphériques du STM32

A. *Configuration GPIO*

Broche	Signal	Mode programmer	Etat / niveau
PA2	USART2_TX	Sortie AF push-pull, 10 MHz	Niveau piloté par l'USART2
PA3	USART2_RX	Entrée flottante	Non fixé par logiciel
PA4	/CS	Sortie push-pull, 10 MHz	0 pendant GPIO_InitPin, puis 1 au repos
PA5	SCK	Sortie AF push-pull, 10 MHz	Repos haut (CPOL=1)
PA6	MISO	Entrée flottante	Défini par le capteur
PA7	MOSI	Sortie AF push-pull, 10 MHz	Piloté par SPI1
PA10	INT1	Entrée flottante	Lecture de IDR
PA11	INT2	Entrée flottante	Lecture de IDR

Figure 6 : Configuration GPIO STM32

B. *Configuration UART2*

Registre	Catégorie	Valeur du C	Commentaire
RCC->APB2ENR	Horloge GPIO	IOPAEN = 1 (bit 2)	Active l'horloge du port A pour PA2/PA3.
GPIOA->CRL	Configuration E/S	PA2 = 0x9 ; PA3 = 0x4	PA2 : AF push-pull 10 MHz ; PA3 : entrée flottante.
RCC->APB1ENR	Horloge USART	USART2EN = 1 (bit 17)	Active le périphérique USART2.
RCC->CFGR	Lecture horloge	PPRE1 lu dans bits [10:8]	Calcule PCLK1 depuis la fréquence passée en argument et le prédiviseur APB1.
USART2->BRR	Débit binaire	0x0EA6 si PCLK1 = 36 MHz	Diviseur 3750 pour 9600 bauds ; calculé dans config_usart2(72 MHz, 9600).
USART2->CR1	Contrôle	UE=1, M=0, PCE=0, TE=1, RE=1	USART actif, 8 bits, sans parité, émission et réception actives.
USART2->CR2	Contrôle	STOP[1:0] = 00	Un bit de stop : format 8N1.
USART2->CR3	Contrôle	CTSE=0, RTSE=0	Pas de contrôle de flux matériel RTS/CTS.
USART2->SR / DR	État / données	TXE testé ; DR alimenté	Envoi d'un octet lorsque le registre d'émission est disponible.

Figure 7 : Configuration périphérique USART2

C. *configuration SPI1*

Le SPI1 est utilisé en mode maître, avec des échanges sur 8 bits, un /CS manuel et le mode SPI 3 (CPOL=1, CPHA=1)

Registre	Catégorie	Valeur du C	Commentaire
RCC->APB2ENR	Horloge périphérique	SPI1EN = 1 (bit 12)	Active l'horloge du SPI1.
RCC->APB2ENR	Horloge GPIO	IOPAEN = 1 (bit 2)	Active le port A des lignes SPI.
GPIOA->CRL	Configuration E/S	PA4=0x1 ; PA5=0x9 ; PA6=0x4 ; PA7=0x9	/CS : sortie ; SCK/MOSI : sorties AF ; MISO : entrée flottante.
SPI1->CR1	Configuration	0x036F après SPE=1	MSTR=1, BR=101 (/64), SSM=1, SSI=1, DFF=0, LSBFIRST=0, CPOL=1, CPHA=1, SPE=1.
SPI1->CR2	Configuration	Non écrit dans le C	Pas de DMA ni d'interruption SPI configurés dans cette fonction.
SPI1->SR	État	RXNE testé (=1)	Attente d'un octet reçu à chaque échange.
SPI1->DR	Données	Adresse/commande puis données	L'envoi sur MOSI génère également la réception sur MISO.
GPIOA->BSRR / BRR	Commande /CS	PA4 = 1 / PA4 = 0	gere_cs(1) termine l'échange ; gere_cs(0) le commence.

Figure 8 : Configuration registre SP1

Paramètre	Valeur
Rôle	Maître (MSTR = 1)
Fréquence SPI	72 MHz / 64 = 1,125 MHz (selon horloge APB2 du projet)
Format	8 bits (DFF = 0) ; bit de poids fort en premier
Mode SPI	Mode 3 : CPOL = 1, CPHA = 1
Gestion NSS	Logicielle : SSM=1, SSI=1 ; /CS sur PA4
Activation	SPE = 1

Opération	Principe appliqué dans le C
Écriture d'un registre	R=0, MB=0 ; commande = adresse & 0x3F, puis valeur.
Lecture d'un registre	R=1, MB=0 ; commande = 0x80 | adresse, puis octet factice.
Lecture multiple X/Y/Z	R=1, MB=1 ; commande = 0xF2 pour DATAX0 (0x32), puis 6 octets.
État de /CS	À 0 pendant la transaction ; à 1 entre deux transactions.

Figure 9 : Parametre et opération lecture/ecriture sur bus SPI

VII. Configuration des registres ADXL345

Valeurs finales après l'exécution de config_ADXL(). Les adresses des registres proviennent de ADXL345.h et les octets écrits sont relevés dans main.c

Registre	Adresse	Valeur finale	Signification
POWER_CTL	0x2D	0x08	Séquence 0x00 puis 0x08 : mesure activée (MEASURE=1).
DATA_FORMAT	0x31	0x0B	SPI 4 fils ; ±16 g ; FULL_RES=1 ; données justifiées à droite ; INT actives haut.
BW_RATE	0x2C	0x0A	Cadence 100 mesures par seconde (100 Hz).
FIFO_CTL	0x38	0x90	FIFO en mode Stream ; seuil WATERMARK à 16 échantillons.
INT_MAP	0x2F	0x01	WATERMARK → INT1 ; OVERRUN → INT2.
INT_ENABLE	0x2E	0x03	Écrit 0x00 puis 0x03 : seules WATERMARK et OVERRUN sont activées.
ACT_INACT_CTL	0x27	0x77	Détection activité/inactivité sur X, Y, Z en mode DC.
THRESH_ACT	0x24	0x10	Seuil activité : 16 × 62,5 mg = 1 g.
THRESH_INACT	0x25	0x08	Seuil inactivité : 8 × 62,5 mg = 0,5 g.
TIME_INACT	0x26	0x01	Durée inactivité : 1 s (pas de 1 s).
THRESH_TAP	0x1D	0x50	Seuil choc paramétré : 80 × 62,5 mg = 5 g.
DUR	0x21	0x10	Durée max. d'un choc : 16 × 0,625 ms = 10 ms.
LATENT	0x22	0x00	Intervalle de latence double choc mis à zéro.
TAP_AXES	0x2A	0x00	Écrit 0x07 puis réécrit 0x00 : axes TAP désactivés en état final.
THRESH_FF	0x28	0x09	Seuil chute libre : 9 × 62,5 mg = 0,5625 g.
TIME_FF	0x29	0x0A	Durée chute libre : 10 × 5 ms = 50 ms.

Figure 10 : Valeurs registre configuré ADX345

VIII. Processing avec l'ADXL345

Dans la boucle principale, le STM32 scrute PA10 (INT1). Lorsque WATERMARK est actif et que la FIFO logicielle possède au moins sept emplacements libres, il lit FIFO_STATUS, récupère une mesure X/Y/Z, puis empile une trame de sept octets. La FIFO TX circulaire contient 32 octets ; l'envoi par USART2 est réalisé sans attente bloquante quand le drapeau TXE est disponible.

Voir figure 5, pour la méthodologie d'envoyé de la trame.

Le format 8N1 utilise 10 bits par octet. À 9600 bauds, on dispose de 960 octets/s, soit 960 / 7 ≈

137 trames de mesure/s en débit théorique continu. À 100 mesures/s, les trames occupent environ 7000 bits/s,

hors éventuels surcoûts logiciels.

Le programme Visualisation_ADXL345.pde recherche l'en-tête 0xF0 puis collecte les six octets suivants. Il reconstitue les valeurs signées sur 13 bits (complément à deux), applique 3,9 mg/LSB et affiche les accélérations en g et m/s^2. L'historique circulaire conserve 650 mesures, soit plus que les 500 demandées. L'interface dispose également d'un mode de démonstration explicitement distinct du port série réel.

Une zone de Processing présente aussi la dernière trame UART reçue ainsi que les octets MISO reconstitués à partir de cette trame :

ces derniers ne doivent pas être confondus avec une mesure directe du bus SPI.

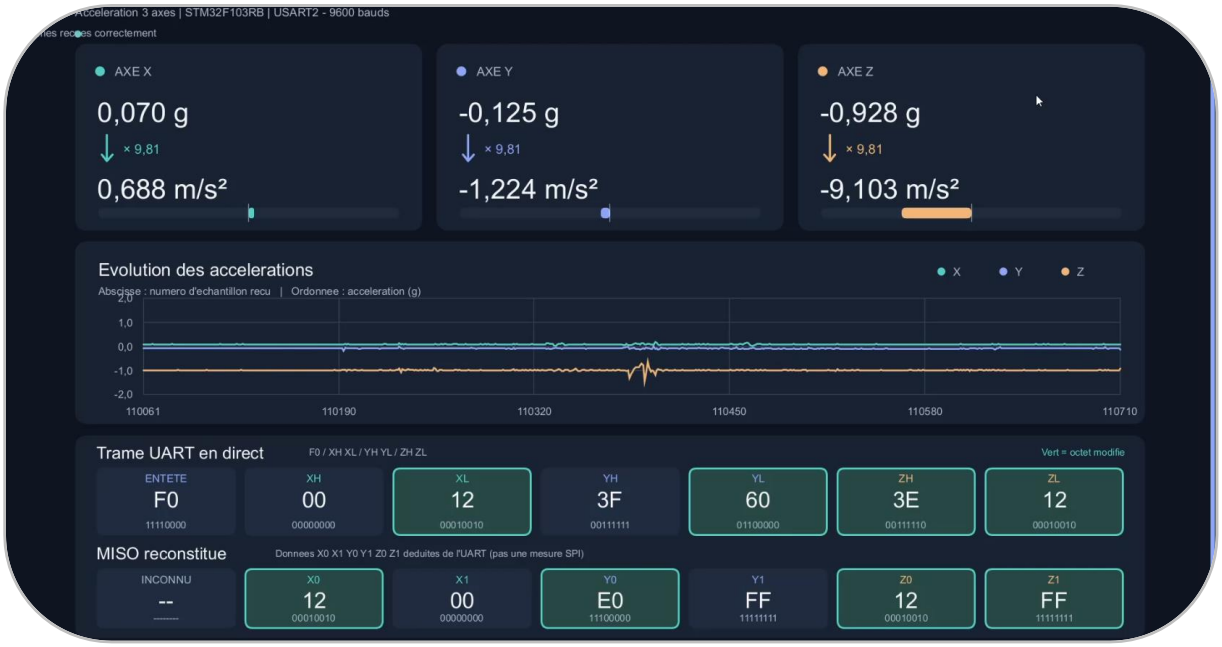


Figure 11 : Capture d'écran logiciel processing

IX. Chronogrammes de l'analyseur logique

A. SPI

Plusieurs acquisitions PulseView jointes à l'archive montrent le bus SPI en fonctionnement. On s'atradra seulement sur deux images pour les explications. Elles permettent de relier les niveaux logiques aux octets transmis.



Figure 12 : Extrait de Data_spi/Message_0.jpg

La commande 0xB9 = 0x80 | 0x39 lit FIFO_STATUS. La réponse utile 0x10 signifie que la FIFO du capteur contient 16 entrées. Ensuite 0xF2 = 0xC0 | 0x32 lance la lecture multi-octets depuis DATAX0. Le premier octet MISO de cette seconde transaction est ignoré : il est reçu pendant l'envoi de l'adresse.

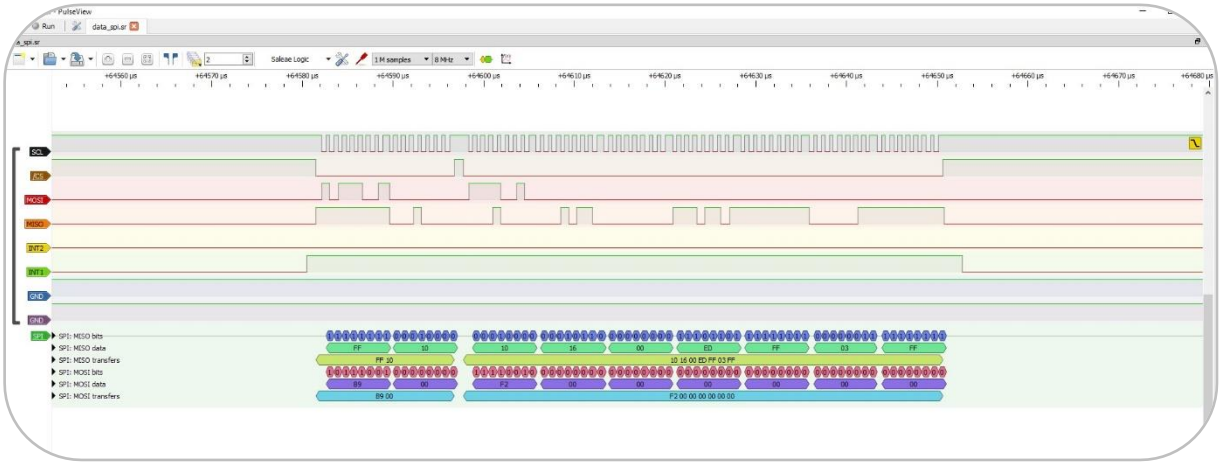


Figure 13 : Extrait de Data_spi/Message_6.jpg

Sur cette capture de l'analyseur logique, on peut observer les échanges SPI entre le STM32 et l'accéléromètre ADXL345. Le STM32 envoie une commande de lecture sur MOSI, puis le capteur renvoie les six octets correspondant aux accélérations des axes X, Y et Z sur MISO.

En comparant les deux trames, on remarque que certains octets changent. Cela montre que les mesures récupérées par le STM32 évoluent au cours du temps. On peut donc vérifier que la communication fonctionne et que le capteur transmet bien ses données.

B. USART

1. Exemple trame USART

Sur cette capture de l'analyseur logique, on peut observer la trame UART envoyée par le STM32 vers l'ordinateur à 9600 bauds. La trame commence par l'en-tête 0xF0, qui permet à Processing de reconnaître le début d'une nouvelle mesure. Les six octets suivants (00 18 3F 6B 3E 02) correspondent aux valeurs des accélérations X, Y et Z, avec deux octets par axe. On peut également distinguer les bits de début (S) et de fin (T) de chaque octet, conformément au format 8N1. Cette capture permet donc de vérifier que le STM32 transmet bien les données dans le format attendu pour être ensuite reconstituées et affichées dans Processing.

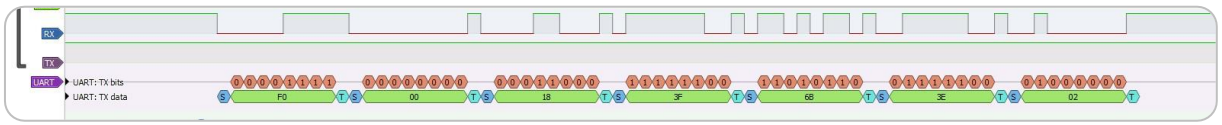


Figure 14 : Démo transmission UART

2. Calcul de la durée de l'impulsion

En lisant l'échelle temporelle du chronogramme, on observe que le début de l'impulsion se situe vers 30 975 µs et que sa fin se situe vers 31 078 µs.

La durée de l'impulsion est donc de 31 078 - 30 975 = 103 µs.

À 9600 bauds, la durée théorique d'un bit est de 1 / 9600, soit environ 104,17 µs.

La différence entre la durée mesurée et la durée théorique est d'environ 1,17 µs. Cette différence reste faible et peut s'expliquer par la précision de lecture de la capture.

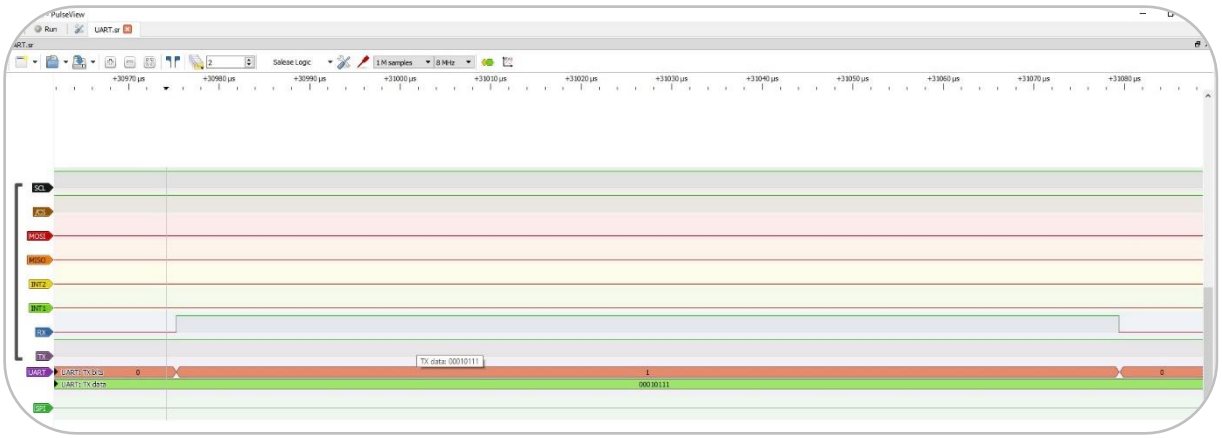


Figure 15 : Un bit sur instant donnée

X. Difficultés rencontrées et assistance IA

Globalement, je n'ai plus l'historique de mes conversations avec l'IA, mais en gros, on débattait. Je lui expliquais comment je voulais faire les choses, selon ma vision, puis elle me disait qu'en fait, vous ne demandiez pas ça, mais plutôt ça. J'essayais alors de comprendre pourquoi, et ainsi de suite.

En vrai, au début, mes prompts étaient vachement bateau : je donnais très peu d'informations, en mode « Fais-moi ça comme ci, comme ça ». Puis, petit à petit, j'ai compris qu'il fallait vraiment essayer de structurer tout ça et de comprendre comment formuler mes demandes pour que l'IA puisse bien m'expliquer les choses et me générer le code.

XI. Dessin demandé

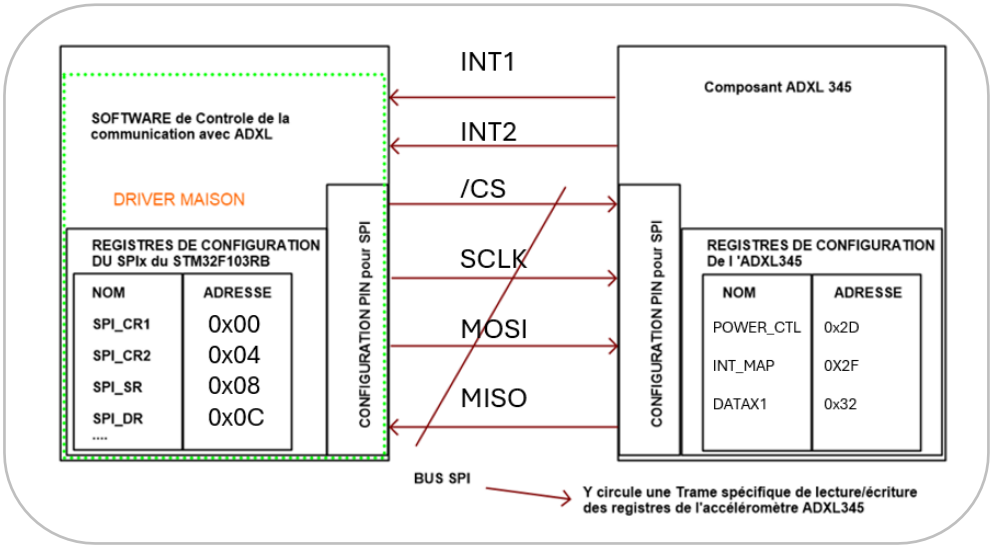


Figure 16 : Dessin demandé

XII. Table des figures

Figure 1 : Visualisation Keil µVision + Visual Studio Code 6
Figure 2 : Présentation du câblages 8
Figure 3 : Bilan E/S - Carte Nucleo STM32 F103RB 9
Figure 4 : Tableau de raccordement analyseur logique 9
Figure 5 ; Format de la trame UART 13
Figure 6 : Configuration GPIO STM32 15
Figure 7 : Configuration périphérique USART2 15
Figure 8 : Configuration registre SP1 16
Figure 9 : Parametre et opération lecture/ecriture sur bus SPI 16
Figure 10 : Valeurs registre configuré ADX345 17
Figure 11 : Capture d'écran logiciel processing 18
Figure 12 : Extrait de Data_spi/Message_0.jpg 19
Figure 13 : Extrait de Data_spi/Message_6.jpg 19
Figure 14 : Démo transmission UART 20
Figure 15 : Un bit sur instant donnée 20
Figure 16 : Dessin demandé 21